Linear Regression

Machine Learning with Python — Lecture 1, Session 2 (Oct 19)

EUGLOH — *Problem Solving Using Open-Source Languages; R and Python*

University of Novi Sad

Today's session

- **The linear model** — a straight line through the data
- **Least squares** — the mean squared error, the closed-form fit, and gradient descent
- The 1801 story of **Piazzi, Ceres and Gauss** — one of the first fits to data
- Interactive demo: how the fit **stabilises** as you draw more random samples

Session 2 of 4 today. *(Logistic regression — the sigmoid, cross-entropy, training, and multi-class — now opens tomorrow's neural-network sessions.)*

Part 1 — Linear regression

The linear model

Our first **supervised** model. With **one feature** it is simply a straight line:

$$\hat{y} = w\,x + b$$

- w — the **weight** (the slope): how much the prediction changes when x goes up by 1
- b — the **bias** (the intercept): the prediction when $x = 0$
- $\hat{y}$ ("y-hat") is our **prediction** — a continuous number (a price, a temperature, …)

With several features the same idea gives a **flat surface** (a *hyperplane*). We keep the one-feature picture, because it is the easiest to see.

The first great fit — Piazzi, Ceres and Gauss (1801)

- On **1 January 1801** Giuseppe Piazzi, an astronomer in Palermo, spotted a faint moving point of light — a new body we now call the dwarf planet **Ceres**.
- He tracked it for **41 nights**, then it vanished into the glare of the Sun. Nobody could find it again.
- **Carl Friedrich Gauss**, then 24, took Piazzi's handful of observations, fitted a model to them with the method of **least squares**, and predicted where Ceres would reappear.
- Astronomers searched — and found Ceres almost exactly where Gauss had said it would be.

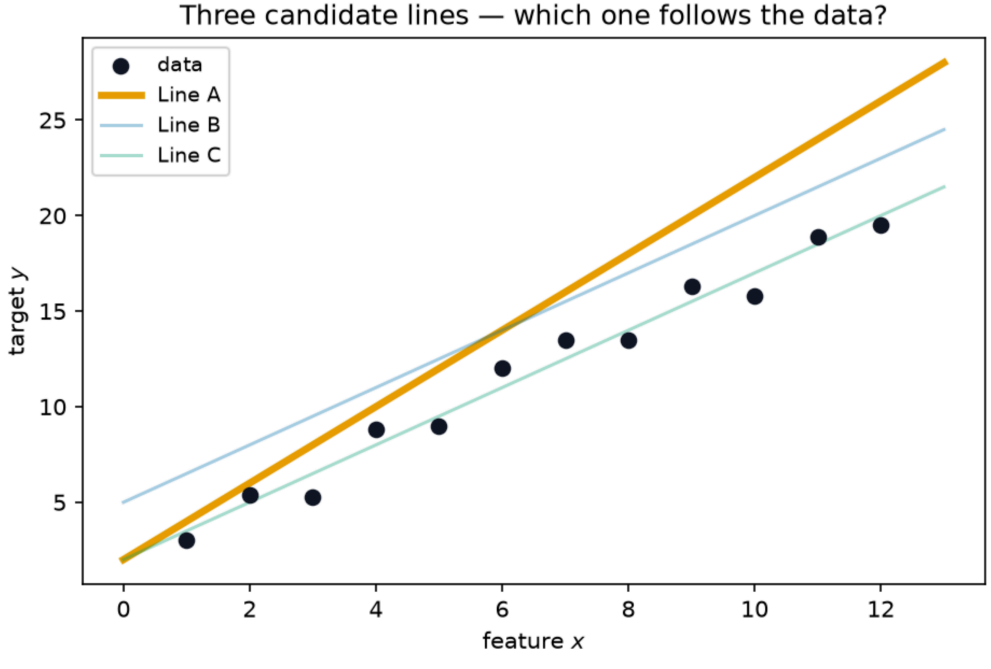
This is often called one of the **first uses of learning from data**: fit a model to the examples you have, then predict the ones you have not seen. The tool was the same **least-squares line** we are about to derive.

Your turn — which line fits?

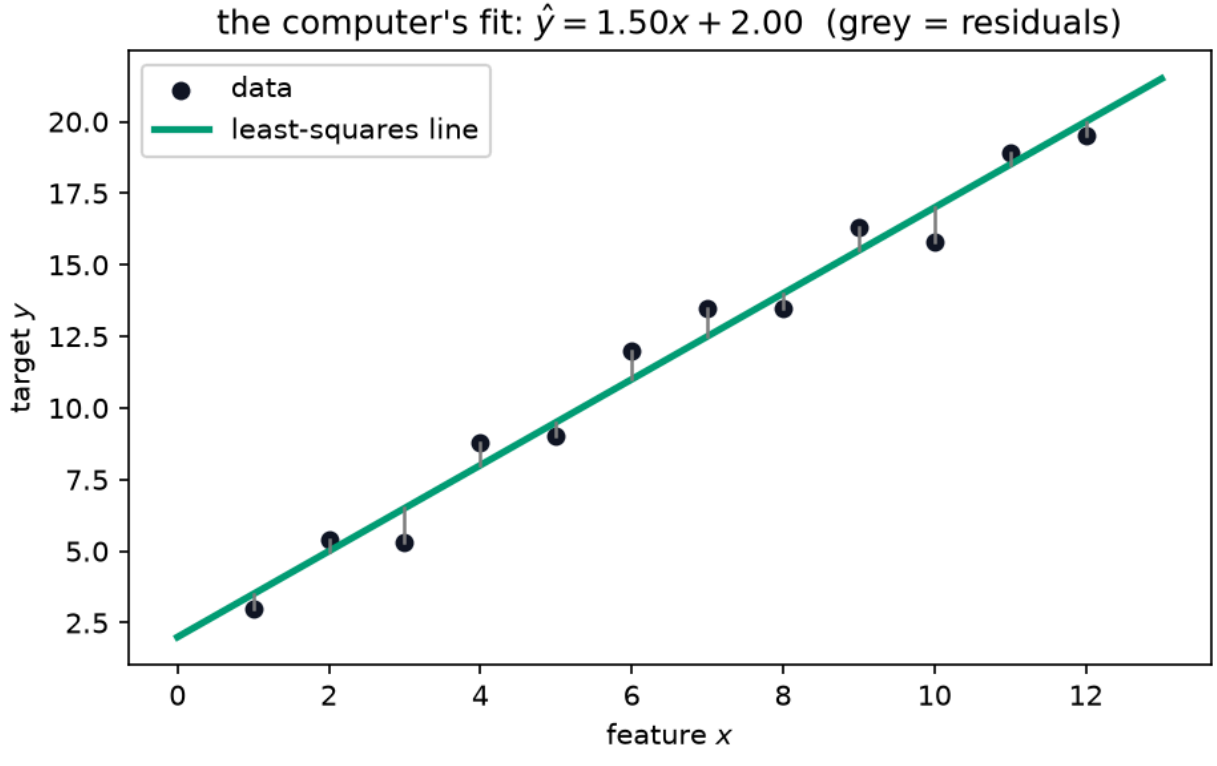
Here are **12 measurements** and **three straight lines**. Two of them miss the data; one follows it. **No formulas yet** — judge by eye, commit to a line, then reveal the verdict.

Which line follows the data best? Reveal the verdict

- (●) Line A
- ( ) Line B
- ( ) Line C



You picked Line A (drawn thicker). Press **Reveal the verdict** to check.


the computer's fit: $\hat{y} = 1.50x + 2.00$ (grey = residuals)
data
least-squares line
target y
feature x
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0
2
4
6
8
10
12

Choosing the line — the mean squared error

We want the line that is *closest* to the data. "Closest" means making the errors small, and we measure them with the **mean squared error** (MSE):

$$\mathcal{L}(w, b) = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2$$

In plain words: for each point take its vertical distance to the line (the **residual** $\hat{y}_i - y_i$), **square** it, and **average** over the n points.

- squaring counts every error positively and punishes large errors much more than small ones
- the squared error is **smooth** and **convex** — it has one lowest point, so there are no local traps
- the grey segments in the figure above are exactly those residuals

Two ways to find the best line

1. The closed form. The MSE is a smooth bowl, so its lowest point can be found with a formula. For one feature the best line is

$$w = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad b = \bar{y} - w\,\bar{x}$$

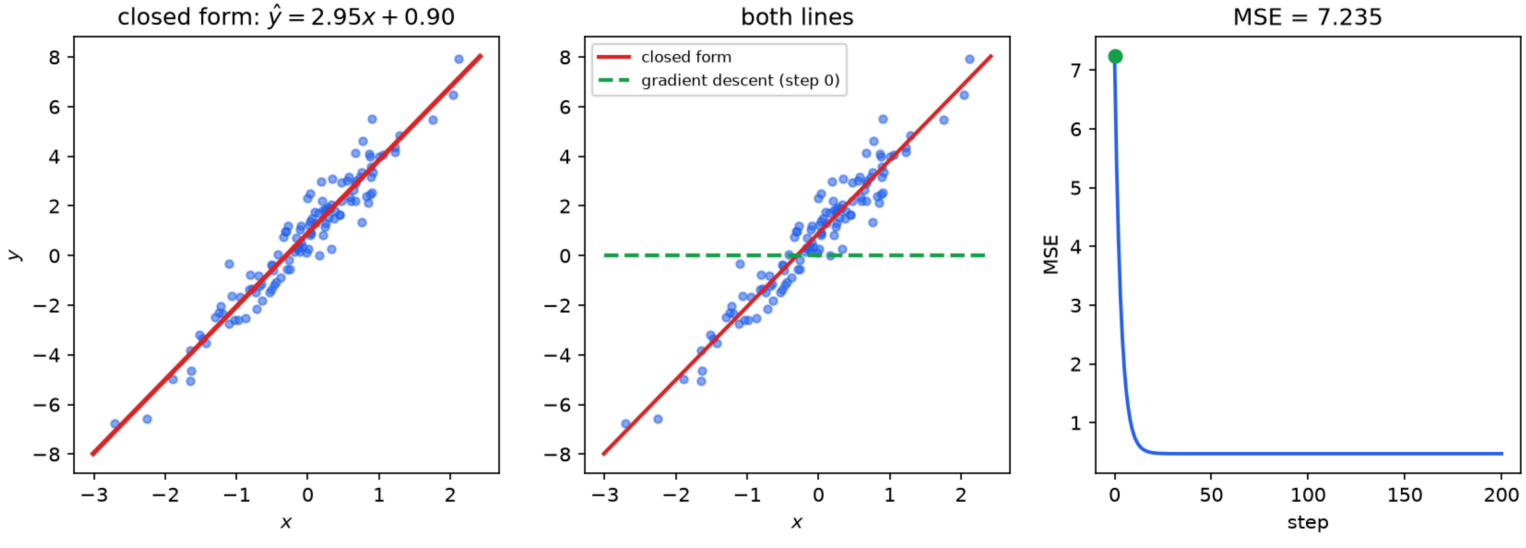
where $\bar{x}$ and $\bar{y}$ are the means.

2. Gradient descent. With very many features that formula is expensive, so instead we *walk downhill* on the loss:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}, \qquad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}$$

- $\partial \mathcal{L} / \partial w$ points in the direction the loss grows; we step the **opposite** way
- η is the **learning rate** — how large each step is
- repeat many times, and the line creeps towards the same answer

gradient-descent step 0

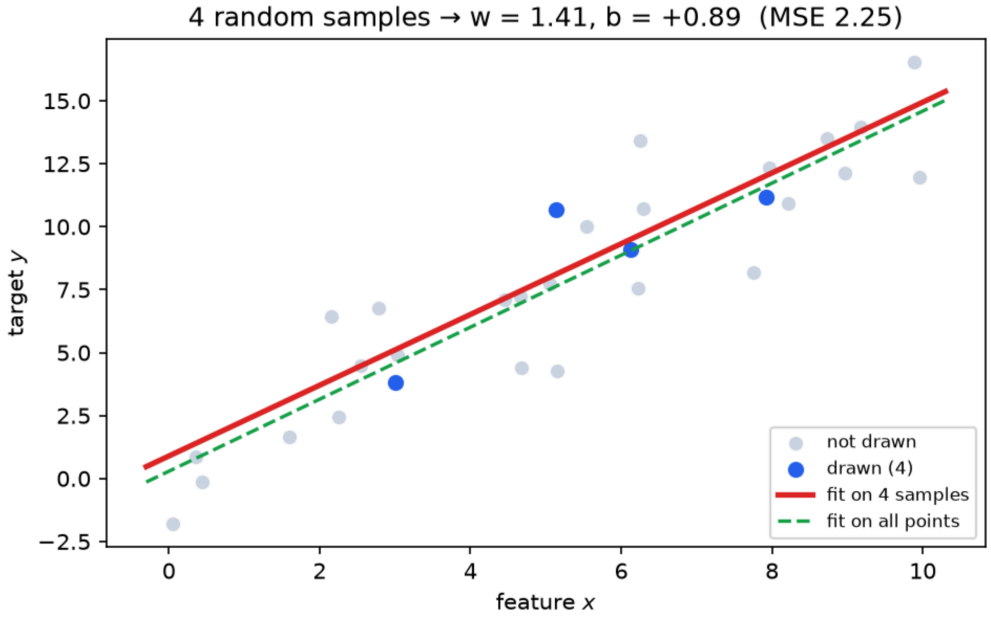


Interactive — fitting a line from random samples

Each time you **draw**, we pick k points **at random** (not the first k) and fit the line to those k alone.

- with **few** points the line jumps around — every draw gives a very different fit
- as k grows the line **settles down** towards the least-squares line for the whole dataset
- drag k and click **draw** a few times to feel the difference

random samples (k) 4 draw a new sample

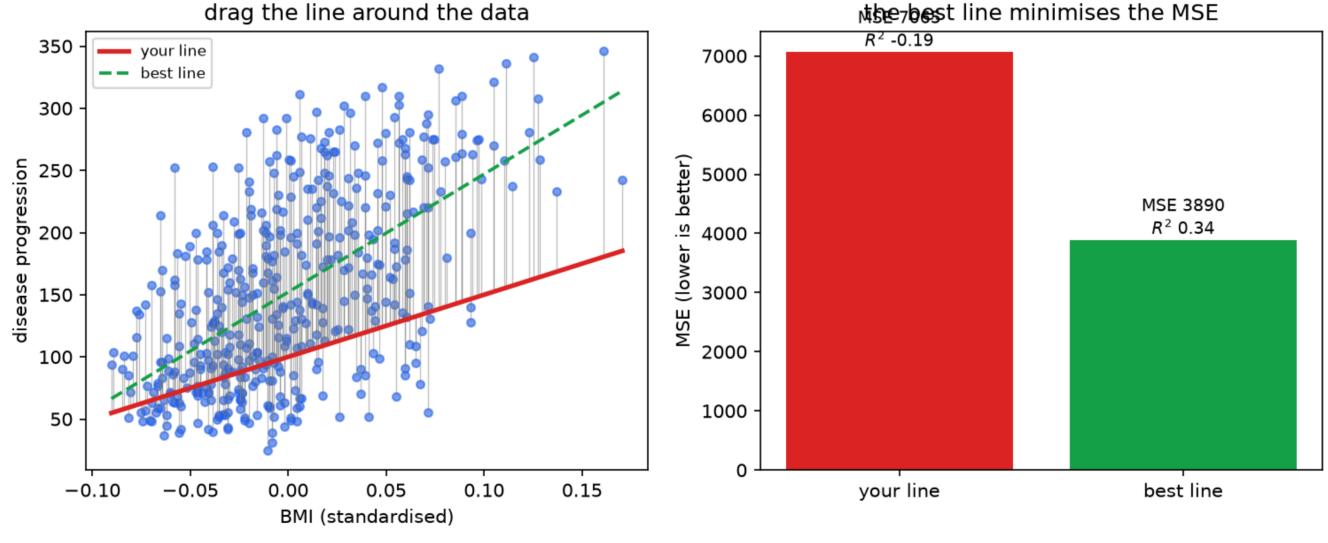


Evaluating a regression model

- **R² (coefficient of determination)** — the fraction of the target variance the model explains: 1 is perfect, 0 means "no better than predicting the mean", and it *can go negative* on a bad model.
- **RMSE** — root mean squared error, in the same units as y, so its magnitude is directly interpretable.

Both numbers are properties of **the line you pick**. Drag the sliders below (diabetes: BMI → disease progression) and watch R^2 and MSE respond — the *best* line is the one that makes the MSE smallest. Report these on a **held-out test set**, never the training set.

slope w 500 intercept b 100



Linear regression in scikit-learn

```
from sklearn.linear_model import LinearRegression, SGDRegressor
model = LinearRegression().fit(X_train, y_train)   # closed form
model.score(X_test, y_test)                         # R²
```

- `LinearRegression` — exact least squares (closed form)
- `SGDRegressor` — gradient descent; scales to very large datasets
- **regularised** cousins: `Ridge` (L2 penalty) and `Lasso` (L1) trade a little bias for a lot less variance — the same idea we will meet again in the neural-network sessions tomorrow.

Summary

- **Linear regression** predicts a number and minimises the **MSE** — solvable in closed form, or by **gradient descent**. (Gauss fitted least squares to Piazzi's Ceres observations back in 1801.)
- We can watch the fit **stabilise as we draw more random samples** — few points give a shaky line, many give a steady one.
- **Always report on a held-out set**; use **R^2** and **RMSE** to judge a regression model.
- **Feature scaling** and **regularisation** are the practical levers.

Next session: **decision trees**. Tomorrow: **logistic regression**, the bridge into neural networks.